

1) Find the directory in `/root/86344/`, where the owner is not the `root` user. Write the full (absolute) path name to the file: `/root/86344.ans`

```
# find /root/86344/ -not -user root  
/root/86344/5/4/1  
# find /root/86344/ -not -user root >/root/86344.ans
```

2) Find the directory in `/root/160171/`. In which the sticky bit is set. Write the full (absolute) path name to the file: `/root/160171.ans`

```
# find /root/160171/ -perm /1000  
# find /root/160171/ -perm /1000 >/root/160171.ans
```

3) Find the folder inside the `/root/17107/` folder, which has an enabled sticky bit. Save the full path of this folder into the `/root/17107.ans` file.

```
find /root/17115 -perm -1000 >/root/17115.ans
```

1) Create a new user with the name `user86315` and lock the password of the user!

```
# useradd user86315
```

```
# passwd -l user86315  
Locking password for user user86315.  
passwd: Success
```

(if you wanna verify if the user account is locked)

```
# passwd -status user86315
```

2) Create a new user with name `user17201` and set that this user must wait minimum 15 days between two password changes, but can only wait a maximum of 20 days.

```
useradd user17201
```

```
chage -m 15 -M 20 user17201
```

3) Create a file with name `/root/86198.dat` and with a size of `861980` bytes.

```
# dd if=/dev/zero of=/root/86198.dat bs=861980 count=1
```

4) Create a new `/dev/sdc1` partition on the drive `/dev/sdc`. Format the partition with `ext2` filesystem and mount it to the `/mnt` folder in read only mode.

```
# fdisk /dev/sdc
```

```
n p w
```

```
# mkfs.ext2 /dev/sdc1
```

```
# mount -o ro /dev/sdc1 /mnt
```

```
# mount
```

5) Create a new `crypto_LUKS` encrypted `xfs` filesystem on `/dev/sdc` and mount it to the `/mnt` directory.

```
# cryptsetup luksFormat /dev/sdc
```

```
# cryptsetup luksOpen /dev/sdc x
```

```
# mkfs.xfs /dev/mapper/x
```

```
# umount /mnt/
```

```
# mount /dev/mapper/x /mnt
```

6) Create a new empty file name : `/root/test-1.txt`

```
touch /root/test-1.txt
```

7) Create a `/dev/sdb1` partition on drive `/dev/sdb`, then prepare this partition for encryption by using the `cryptsetup` program and the LUKS format.

```
fdisk /dev/sdb -> n -> defaults -> w
```

```
cryptsetup luksFormat /dev/sdb1 -> YES
```

```
(password)
```

8) Create the `user160184` user and generate a new default ssh RSA key (`/home/user160184/.ssh/id_rsa`) for it without passphrase. Allows `user1608184` to log as `root` with the help of the RSA private key.

```
# useradd user160184 /useradd: /creating user
```

```
# su user160184 //su: change the user
```

```
$ ssh-keygen //generate a new default ssh RSA key
```

```
(May be) ssh-keygen -t rsa
```

```
$ ssh-copy-id root@localhost //allows to user16084 to log s root
//choose yes
//type password "asd"
$ exit
```

9) Create a private-public RSA key pair with the original settings with names /root/26636.pem and /root/26636.pub, then encrypt the /root/26636.txt file by using the private key. Save the encrypted file as /root/26636.enc.

```
openssl genrsa -out /root/26636.pem
openssl rsa -in /root/26636.pem -pubout -out /root/26636.pub | touch 22636.txt
openssl dgst -sign /root/26636.epm -out /root/26636.sign /root/26565.enc
```

10) Create a folder named /root/26602 and set its ACL such that user-26602 will automatically get every permissions (read-write-execute) to its SUBFOLDERS and SUBFILES. Do not change permissions for any other users.

```
mkdir /root/26602 setfacl -dm user-26602:rwX /root/26602
```

11) Create a private-public RSA key pair with names /root/26595.pem and /root/26595.pub and sign the /root/26595.txt file with these keys. The signature must be named /root/26595.sign#.

```
openssl genrsa -out /root/26595.pem
openssl rsa -in /root/26595.pem -pubout -out /root/26595.pub
openssl dgst -sign /root/26595.pem -out /root/26595.sign /root/26595.txt
```

12) Create a user named user-26552.

```
useradd user-26552
```

13) Create a self-signed certificate for the domain (CN) my-comapny.eu and save the certificate to the file: /root/my-company.crt.

```
openssl req -newkey rsa:2048 -nodes -keyout /root/my-company.key -x509 -days 365 -out /root/my-company.crt -subj "/CN=my-company.eu"
```

1) Who signed the file /root/contract-86302.txt, Anne, Bob, Carol, Daniel or Eve?

The signature was created with the help of the SHA 256 hash algorithm and stored in the file: /root/contract-86302.sha256.signature

The public key of Anne is in `/root/Anne-86302.pub.pem`

the public key of Bob is in `/root/Bob-86302.pub.pem`

the public key of Carol is in `/root/Carol-86302.pub.pem`

the public key of Daniel is in `/root/Daniel-86302.pub.pem`

the public key of Eve is in `/root/Eve-86302.pub.pem`

Write the answer (the name) to the file: `/root/86302.ans`

```
# openssl dgst -verify /root/Anne-86302.pub.pem -signature /root/contract-86302.sha256.signature /root/contract-86302.txt
```

Verification Failure

```
# openssl dgst -verify /root/Bob-86302.pub.pem -signature /root/contract-86302.sha256.signature /root/contract-86302.txt
```

Verification Failure

```
# openssl dgst -verify /root/Carol-86302.pub.pem -signature /root/contract-86302.sha256.signature /root/contract-86302.txt
```

Verified OK

```
# openssl dgst -verify /root/Daniel-86302.pub.pem -signature /root/contract-86302.sha256.signature /root/contract-86302.txt
```

Verification Failure

```
# openssl dgst -verify /root/Eve-86302.pub.pem -signature /root/contract-86302.sha256.signature /root/contract-86302.txt
```

Verification Failure

```
# echo Carol > /root/86302.ans
```

1) Decrypt the `/root/86261.enc` file by using 86261 as the key and 0 as the initialization vector. The file was encrypted by using the `aes-128-cbc` algorithm. Save the result as `/root/86261.txt`.

```
# openssl aes-128-cbc -d -in /root/86261.enc -K 86261 -iv 0 > /root/86261.txt
```

```
# cat /root/86261.txt
```

2) Decrypt the `/root/86257.enc` file by using the `86257` password and the des-edc algorithm. Save the result as `/root/86257.txt`.

```
# openssl des-edc -d -in /root/86257.enc -k 86257
```

```
*# openssl des-edc -d -in /root/86257.enc -k 86257 > /root/86257.txt
```

```
*# cat /root/86257.txt
```

3) Decrypt the `/root/17090.enc` file which was encrypted by using the following privatekey. Save the result as `/root/17090.txt`!

```
# openssl des-edc -d -in /root/86257.enc -k 86257
```

```
# openssl des-edc -d -in /root/86257.enc -k 86257 > /root/86257.txt
```

```
# cat /root/86257.txt
```

1) Encrypt the file located at `/root/17083.txt` by using the `/root/17083.key.pub` public key. Save the encrypted file as `/root/17083.enc`!

```
openssl rsautl -encrypt -inkey /root/17083.key.pub -pubin -in /root/17083.txt -out /root/17083.enc
```

2) Encrypt the `/root/26582.txt` file by using DES CBC with the key `0123456789ABCDEF` and with initialization vector `0`. The name of the output file should be `/root/26582.enc`.

```
openssl des-cbc -K 0123456789ABCDEF -iv 0 -in /root/26582.txt -out /root/26582.enc
```

3) Encrypt the file `/root/113937.txt` by using the `113937` password and the des-edc algorithm. Save the result as `/root/113937.des-edc` (do not change the default key generation method) (1)

```
openssl des-edc -in /root/113937.txt -k 113937 -out /root/113937.des-edc
```

1) Set the permissions of the `/root/86237.txt` file to read-only.

```
# chmod 444 /root/86237.txt
```

```
# stat /root/86237.txt //stat: gives file and filesystem
```

2) Set the permissions of the /root/26673.txt file to read only with the sticky bit enabled.

```
chmod 2400 /root/26673.txt
```

3) Set the permissions of the file /root/160101.txt so that the permissions show as -r-rws-T in the directory listing.

```
# chmod 3470 /root/160101.txt
```

4) Set the permissions of the file /root/113924.txt so that its permissions shows as -r-Srwx---- in directory listing. (1)

```
chmod 1570 /root/113924.txt
```

5) Set owner of the /root/17159.txt file to the operator user. (Do not change anything else about the file.)

```
# chown operator /root/86211.txt
```

6) Set that user-26660 has to change his/her password at most every 5 days.

```
chage -m 5 user-26660
```

7) Set ownership of the file /root/160076.txt so that the owner will be operator and the owner group will be users (do not change anything else about this file)

```
# chown operator:users /root/160076.txt
```

8) Set that user-26626 has read and write permissions on the /root/26626.txt file. Do not change any other permissions to any other users.

```
setfacl -m user-26626:rw /root/26626.txt
```

9) Set the owner of the /root/26565.txt file to user-26565.

```
chown user-26565 /root/26565.txt
```

10) Set the expiration date of the user account operator to 2022-01-20 (1)

```
chage -E 2022-01-20 operator
```

11) Set owner group of the /root/113898.txt file to users (Do not change anything else about the file) (1)

```
chgrp users /root/113898.txt
```

12) Set the owner of all the files and directories under the /opt/265260/ directory to operator recursively.

```
sudo chown -R operator /opt/265260/
```

1) Give read-only permissions to the operator user on the /root/86224.txt file with a new USER ACL entry. (Do not change anything else about the file and do not give or take any other permissions.)

```
# setfacl -m u:operator:r /root/86224.txt
```

```
# getfacl /root/86224.txt
```

1) Download the file 5d3a8e1b with the help of the command below.

```
curl -o /5d3a8e1b https://progcont.hu/docs/IT-Security/loop/5d3a8e1b
```

The file contains an encrypted filesystem in LUKS format. The password is: 5d3a8e1b. Copy the content of the filesystem to the directory: /root

```
# curl -o /5d3a8e1b https://progcont.hu/docs/IT-Security/loop/5d3a8e1b
```

```
# losetup -f /5d3a8e1b
```

```
# lsblk -f
```

```
# cryptsetup luksOpen /dev/loop0 loop
```

```
Enter passphrase for /5d3a8e1b: 5d3a8e1b
```

```
# mount /dev/mapper/loop /mnt
```

```
# ls /mnt
```

```
070f  lost+found
```

```
**# cp /mnt/070f /root
```

```
**# cp -r /mnt/lost+found/ /root
```

1) Sign the `/root/86247.txt` file with the following private key. Use the SHA 256 hash function. Save the signature as `/root/86247.sign`!

```
# umask 077
```

```
# cat >k
```

```
# openssl dgst -sign k -sha256 -in /root/86247.txt -out /root/86247.sign
```

```
# openssl dgst -sign k -sha256 /root/86247.txt -out /root/86247.sign
```

```
# openssl dgst -sign k -sha256 -out /root/86247.sign /root/86247.txt
```

```
# xxd /root/86247.sign
```

```
// openssl dgst -sign /root/private_key -sha256 -out /root/86247.sign /root/86247.txt  
xxd /root/86247.sign//
```

```
//
```

```
Copy the private key from the question.
```

```
cat > 45439key
```

```
Paste the private key into the terminal.
```

```
Press Enter.
```

```
Press Ctrl+D twice (once to end the input and once to save the file).
```

```
After completing these steps, the private key will be saved in the file named 45439key
```

```
openssl dgst -sign 45439key -sha256 -out /root/45439.sign /root/45439.txt //
```

2) Sign the certificate signature request stored in `/root/my-factory.csr` using the certificate stored in `/root/my-root-ca.crt` and the corresponding private key stored in `/root/my-root-cakey`. Save the signed certificate to the file: `/root/my-factory.crt`.

```
openssl x509 -req -in /root/my-factory.csr -CA /root/my-root-ca.crt -CAkey /root/my-root-cakey -CAcreateserial -out /root/my-factory.crt -days 365
```

1) Copy the first `70` byte of the file `/root/160067.dat` to the file `/root/160067.part`.


```
# dd if=/root/160067.dat of=/root/160067.part bs=70 count=1
```

1) Allow the owner of the following private key to log in as the **root** user.

```
# cat >kk
```

```
# umask
```

```
# mkdir .ssh
```

```
# ssh-keygen -y -f kk >.ssh/authorized_keys
```

```
# stat .ssh
```

```
# stat .ssh/authorized_keys
```

1) Generate an 1024 bit RSA key and save it as **/root/86191.pem** as a **PKCS8** private key

```
# openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2014 -out /root/86191.pem
```

```
# openssl rsa -text -noout -in /root/86191.pem
```

```
* # openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:1024 -out /root/86191.pem
```

2) generate an 2048 bit RSA key and save it as **/root/45383.pem** as **PKCS8** private key

```
# openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out /root/45383.pem
```

1) Configure the user account **operators** so that its password expires on **2021-07-24** and its maximum number of days between password change is **100**.

```
# chage operator -M 100 //chage: change password and set expiration
```

```
# chage operator -l
```

```
# change operator -l
```

```
# change operator -d 2021-04-01
```

```
# change operator -l
```

```
# change operator -d 2021-04-15
```

```
# change operator -l
```

1) Delete all ACL entries of file `/root/160093.txt`

```
# setfacl -b /root/160093.txt
```

1) PKCS8 thing

```
(copy privateKey to file)
```

```
chmod 400 (keyfile)
```

```
ssh-keygen -f (keyfile) -e -m pkcs8
```

```
mkdir /root/.ssh
```

```
ssh-keygen -f public -i -m pkcs8 >/root/.ssh/authorized_keys
```

1) By using masking set that every new file and folder will be created with 0640 permissions.

```
Umask 027
```

1) Add user-26653 to group-26653.

```
usermod user-26653 -g group-26653
```

1) Change the UID of user-26643 to 1234.

```
usermod -u 1234 user-26643
```

2) Change the home folder of user-26578 to /home/U26578. Create the folder if it does not exist.

```
usermod user-26578 -d U26578
```

1) Prepare the /root/26616.img for encryption by mounting it as a loop device then formatting it to LUKS format.

```
losetup -f /root/26616.img
```

```
lsblk -f
```

```
cryptsetup luksFormat /dev/loop0
```

1) Allow the root user to log in as user-26612 by using SSH without using or changing the password of user-26612. Do not change the default settings during the key- generation phase

```
ssh-keygen
```

```
ssh-keygen -f ~/.ssh/id_rsa.pub -e -m PKCS8 > /tmp/root_pkcs8.pub
```

```
ssh user-26612@<remote_host>
```

```
mkdir -p ~/.ssh
```

```
chmod 700 ~/.ssh
```

```
cat /tmp/root_pkcs8.pub >> ~/.ssh/authorized_keys
```

```
chmod 600 ~/.ssh/authorized_keys
```

```
exit
```

1) Unlock the following user account: user265277

```
sudo passwd -u user265277
```

1) Create a file

```
touch /root/86302.ans
```